1. 图片的显示

借助QLabel显示的。

Qlabel的基础功能： 显示一串文本，QLabel 控件上还可以放置图片、超链接、动画等内容。

1.1 静态图显示

使用QLabel中的setPixmap函数实现静态图展示

图标icon设置，使用QtGui中的QIcon，实现图标的设置

1.2 动态图显示

使用QLabel中的QMovie函数实现

```
需求：点击按钮后分别显示静态图与动态图，设置界面的icon图标

需要的组件： 按钮 QPushButton，标签 QLabel

实现方式：使用Qt Designer工具完成界面开发，并且实现功能。
```

1.3 按钮背景图片设置以及界面背景图片的设置

```
需求：给窗口添加背景图片，给按钮添加背景图片

需要的组件： 按钮 QPushButton，标签 QLabel

实现方式：手写代码实现功能。
```

华清远见元宇宙实验中心
yzlab.com.cn

给按钮设置背景图片使用 样式表(setStyleSheet)

给界面设置背景图片使用 调色板（使用QPalette类）

QPalette类，用于管理图形用户界面中的颜色和字体。

华清远见
HQYJ.COM

2 总结

2.1 QIcon

在 PyQt5 中，`QIcon` 是一个用于表示图标的类。它属于 `PyQt5.QtGui` 模块，主要用于为按钮、菜单项、工具栏等控件设置图标。`QIcon` 支持多种图像格式（如 PNG、JPEG、SVG 等），并且可以根据不同的状态（如正常状态、禁用状态、选中状态等）显示不同的图标。

`QIcon` 的主要作用是为 GUI 控件提供图标支持。它可以用于以下场景：

- **为按钮设置图标**：如 `QPushButton`、`QToolButton` 等。
- **为菜单项设置图标**：如 `QAction` 在菜单或工具栏中的图标。
- **为窗口设置图标**：如窗口的标题栏图标。
- **根据状态显示不同图标**：如正常状态、禁用状态、选中状态等。

2.1.1 `QIcon` 的导入方式

```
from PyQt5.QtGui import QIcon
```

2.1.2 常用方法

2.1.2.1 创建 `QIcon` 对象

- `QIcon(str filename)`：通过文件路径创建一个 `QIcon` 对象。

  ```
  icon = QIcon("icon.png")
  ```

- `QIcon(QPixmap pixmap)`：通过 `QPixmap` 对象创建一个 `QIcon` 对象。

华清远见元宇宙实验室
yyzlab.com.cn

```
from PyQt5.QtGui import QPixmap

pixmap = QPixmap("icon.png")
icon = QIcon(pixmap)
```

2.1.2.2 为不同状态设置图标

- `addFile(str filename, QSize size, QIcon.Mode mode=QIcon.Normal, QIcon.State state=QIcon.Off)`：为不同的状态和大小添加图标。
  - `filename`：图标文件的路径。
  - `size`：图标的大小（可选）。
  - `mode`：图标的模式，常用的有：
    - `QIcon.Normal`：正常状态。
    - `QIcon.Disabled`：禁用状态。
    - `QIcon.Active`：激活状态（如鼠标悬停时）。
    - `QIcon.Selected`：选中状态。
  - `state`：图标的选中状态，常用的有：
    - `QIcon.Off`：未选中状态。
    - `QIcon.On`：选中状态。

```
icon.addFile("normal_icon.png", QSize(32, 32), QIcon.Normal, QIcon.Off)
icon.addFile("disabled_icon.png", QSize(32, 32), QIcon.Disabled, QIcon.Off)
```

2.1.2.3 获取图标的大小

- `availableSizes(QIcon.Mode mode=QIcon.Normal, QIcon.State state=QIcon.Off)`：获取指定状态下的可用图标大小。

```
sizes = icon.availableSizes(QIcon.Normal, QIcon.Off)
print(sizes)  # 输出可用的图标大小列表
```

2.1.2.4 判断图标是否为空

- `isNull()`：检查 `QIcon` 是否为空（即没有设置任何图标）。

```
if icon.isNull():
    print("图标为空")
```

2.1.2.5 将图标应用于控件

- `QPushButton.setIcon(QIcon)`：为按钮设置图标。

```
button.setIcon(icon)
```

- `QAction.setIcon(QIcon)`：为菜单项或工具栏项设置图标。

```
action.setIcon(icon)
```

- `QWidget.setWindowIcon(QIcon)`：为窗口设置标题栏图标。

华清远见元宇宙实验中心 yyzlab.com.cn

```
window.setWindowIcon(icon)
```

2.1.3 典型应用场景

- **为按钮设置图标**：可以为按钮（如 `QPushButton`）设置图标，增强用户体验。
- **为菜单项设置图标**：可以在菜单或工具栏中使用图标来标识不同的操作。
- **为窗口设置图标**：可以为窗口的标题栏设置图标，提升应用程序的视觉效果。
- **根据状态显示不同图标**：可以根据控件的状态（如正常、禁用、选中等）显示不同的图标。

2.2 QPixmap

在 PyQt5 中，`QPixmap` 是一个用于处理图像的类。它属于 `PyQt5.QtGui` 模块，主要用于加载、显示和操作图像。`QPixmap` 是一个非常高效的图像类，特别适合用于在 GUI 应用程序中显示图像。

`QPixmap` 的主要作用是加载和显示图像。它可以用于以下场景：

- **为控件设置背景图片**：如为窗口、按钮等控件设置背景图片。
- **在标签中显示图片**：可以使用 `QLabel` 来显示 `QPixmap` 图像。
- **图像缩放和裁剪**：可以对图像进行缩放、裁剪等操作。
- **与其他图像类交互**：如与 `QImage` 类进行转换，以便进行更复杂的图像处理。

2.2.1 `QPixmap` 的导入方式

```
from PyQt5.QtGui import QPixmap
```

2.2.2 常用方法

2.2.2.1 加载图像

- `QPixmap(str filename)`：通过文件路径加载图像。

  ```
  pixmap = QPixmap("image.png")
  ```

- `QPixmap.load(str filename)`：从文件加载图像。

  ```
  pixmap = QPixmap()
  pixmap.load("image.png")
  ```

2.2.2.2 图像缩放

- `scaled(QSize size, Qt.AspectRatioMode aspectRatioMode=Qt.IgnoreAspectRatio, Qt.TransformationMode transformMode=Qt.FastTransformation)`：返回一个按指定大小缩放后的 `QPixmap` 对象。
  - `size`：目标大小。
  - `aspectRatioMode`：缩放时是否保持宽高比，常用的有：
    - `Qt.IgnoreAspectRatio`：忽略宽高比。
    - `Qt.KeepAspectRatio`：保持宽高比。
    - `Qt.KeepAspectRatioByExpanding`：保持宽高比并扩展以填充目标区域。
  - `transformMode`：缩放时的变换模式，常用的有：
    - `Qt.FastTransformation`：快速但质量较低的缩放。

华清远见元宇宙实验中心 yyzlab.com.cn

- `Qt.SmoothTransformation`：较慢但质量较高的缩放。

```
scaled_pixmap = pixmap.scaled(QSize(100, 100), Qt.KeepAspectRatio,
Qt.SmoothTransformation)
```

2.2.2.3 图像裁剪

- `copy(QRect rect)`：返回一个裁剪后的 `QPixmap` 对象。
  - `rect`：裁剪区域的矩形（`QRect`）。

```
cropped_pixmap = pixmap.copy(QRect(50, 50, 100, 100))  # 裁剪 (50, 50) 到
(150, 150) 的区域
```

2.2.2.4 保存图像

- `save(str filename, str format=None, int quality=-1)`：将 `QPixmap` 保存为图像文件。
  - `filename`：保存的文件名。
  - `format`：图像格式（如 "PNG"、"JPEG" 等），如果未指定，则根据文件扩展名自动推断。
  - `quality`：保存图像的质量（仅适用于 JPEG 格式），范围为 0 到 100，默认值为 -1（使用默认质量）。

```
pixmap.save("output.png", "PNG")
```

2.2.2.5 获取图像信息

- `size()`：获取图像的大小，返回一个 `QSize` 对象。

```
size = pixmap.size()
print(f"图像大小: {size.width()}x{size.height()}")
```

- `isNull()`：检查 `QPixmap` 是否为空（即没有加载任何图像）。

```
if pixmap.isNull():
    print("图像为空")
```

2.2.2.6 将 `QPixmap` 转换为 `QImage`

- `toImage()`：将 `QPixmap` 转换为 `QImage`，以便进行更复杂的图像处理。

```
image = pixmap.toImage()
```

2.2.2.7 将 `QImage` 转换为 `QPixmap`

- `fromImage(QImage image, Qt.ImageConversionFlags flags=Qt.AutoColor)`：将 `QImage` 转换为 `QPixmap`。

```
pixmap = QPixmap.fromImage(image)
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.2.3 典型应用场景

- **为控件设置背景图片**：可以为窗口、按钮等控件设置背景图片。
- **在标签中显示图片**：可以使用 `QLabel` 来显示 `QPixmap` 图像。
- **图像缩放和裁剪**：可以对图像进行缩放、裁剪等操作。
- **保存和加载图像**：可以从文件加载图像，并将其保存为不同的格式。
- **与其他图像类交互**：可以与 `QImage` 类进行转换，以便进行更复杂的图像处理。

2.3 QMovie

在 PyQt5 中，`QMovie` 是一个用于处理动画图像（如 GIF）的类。它属于 `PyQt5.QtGui` 模块，主要用于加载和播放动态图像文件（如 GIF 动画）。`QMovie` 可以与 `QLabel` 结合使用，以便在 GUI 应用程序中显示动画。

`QMovie` 的主要作用是加载和播放动态图像文件（如 GIF），并将其显示在 GUI 控件中。它可以用于以下场景：

- **播放 GIF 动画**：可以在窗口中播放 GIF 动画。
- **控制动画播放**：可以控制动画的播放、暂停、停止等操作。
- **调整动画速度**：可以调整动画的播放速度。
- **获取动画信息**：可以获取动画的帧数、尺寸等信息。

2.3.1 `QMovie` 的导入方式

```
from PyQt5.QtGui import QMovie
```

2.3.2 常用方法

2.3.2.1 加载动画

- `QMovie(str filename)`：通过文件路径加载动画。

  ```
  movie = QMovie("animation.gif")
  ```

- `setFileName(str filename)`：设置动画文件的路径。

```
movie.setFileName("animation.gif")
```

2.3.2.2 控制动画播放

- `start()`：开始播放动画。

  ```
  movie.start()
  ```

- `stop()`：停止播放动画。

  ```
  movie.stop()
  ```

- `pause()`：暂停动画播放（需要 Qt 5.10 或更高版本）。

  ```
  movie.pause()
  ```

华清远见元宇宙实验中心 yyzlab.com.cn

2.3.2.3 调整动画速度

- `setSpeed(int percent)`：设置动画的播放速度，参数为百分比（默认值为 100，表示正常速度）。

```
movie.setSpeed(200)  # 将播放速度提高到 2 倍
```

2.3.2.4 获取动画信息

- `frameCount()`：返回动画的总帧数。

```
frame_count = movie.frameCount()
print(f"动画总帧数: {frame_count}")
```

- `currentFrameNumber()`：返回当前播放的帧号。

```
current_frame = movie.currentFrameNumber()
print(f"当前帧号: {current_frame}")
```

- `nextFrameDelay()`：返回下一帧的延迟时间（以毫秒为单位）。

```
delay = movie.nextFrameDelay()
print(f"下一帧延迟: {delay} 毫秒")
```

- `isValid()`：检查动画是否有效（即是否成功加载了动画文件）。

```
if movie.isValid():
    print("动画加载成功")
```

2.3.2.5 将动画应用到控件

- `QLabel.setMovie(QMovie)`：将动画设置到 `QLabel` 中进行显示。

```
label.setMovie(movie)
```

2.3.3 典型应用场景

- **播放 GIF 动画**：可以在窗口中播放 GIF 动画，增强用户体验。
- **控制动画播放**：可以控制动画的播放、暂停、停止等操作。
- **调整动画速度**：可以根据用户需求调整动画的播放速度。
- **获取动画信息**：可以获取动画的帧数、尺寸等信息，便于进一步处理。

2.4 QPalette

在 PyQt5 中，`QPalette` 是一个用于管理控件颜色和外观的类。它属于 `PyQt5.QtGui` 模块，主要用于定义 GUI 控件的颜色方案。通过 `QPalette`，你可以自定义控件的背景色、前景色（文本颜色）、选中颜色等。

`QPalette` 的主要作用是为控件提供颜色方案，允许你自定义控件的外观。它可以用于以下场景：

- **设置控件的背景色和前景色**：如窗口背景色、按钮文本颜色等。

华清远见元宇宙实验中心 yyzlab.com.cn

- **自定义控件的选中颜色**：如文本选中时的高亮颜色。
- **支持不同的控件状态**：如正常状态、禁用状态、选中状态等。

2.4.1 `QPalette` 的导入方式

```
from PyQt5.QtGui import QPalette
```

2.4.2 常用方法

2.4.2.1 设置颜色

- `setColor(QPalette.ColorRole role, QColor color)`：为指定的角色设置颜色。
  - `role`：颜色角色，常用的有：
    - `QPalette.Window`：窗口背景色。
    - `QPalette.WindowText`：窗口上的文本颜色。
    - `QPalette.Base`：文本输入框的背景色。
    - `QPalette.Text`：文本输入框中的文本颜色。
    - `QPalette.Button`：按钮的背景色。
    - `QPalette.ButtonText`：按钮上的文本颜色。
    - `QPalette.Highlight`：选中项的背景色。
    - `QPalette.HighlightedText`：选中项的文本颜色。
  - `color`：要设置的颜色，使用 `QColor` 对象。

```
from PyQt5.QtGui import QPalette, QColor

palette = QPalette()
palette.setColor(QPalette.Window, QColor(200, 200, 200))  # 设置窗口背景色为灰色
palette.setColor(QPalette.WindowText, QColor(0, 0, 0))    # 设置窗口文本颜色为黑色
```

2.4.2.2 获取颜色

- `color(QPalette.ColorRole role)`：获取指定角色的颜色。

```
window_color = palette.color(QPalette.Window)
print(f"窗口背景色: {window_color.name()}")
```

2.4.2.3 设置不同状态下的颜色

- `setColor(QPalette.ColorGroup group, QPalette.ColorRole role, QColor color)`：为指定的状态和角色设置颜色。
  - `group`：颜色组，常用的有：
    - `QPalette.Active`：控件处于活动状态时的颜色。
    - `QPalette.Disabled`：控件处于禁用状态时的颜色。
    - `QPalette.Inactive`：控件处于非活动状态时的颜色。
  - `role`：颜色角色。
  - `color`：要设置的颜色。

```
palette.setColor(QPalette.Disabled, QPalette.WindowText, QColor(128, 128, 128))  # 设置禁用状态下的文本颜色为灰色
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.4.2.4 将调色板应用到控件

- `QWidget.setPalette(QPalette)`：将调色板应用到控件。

```
widget.setPalette(palette)
```

2.4.2.5 获取当前控件的调色板

- `QWidget.palette()`：获取控件当前的调色板。

```
current_palette = widget.palette()
```

2.4.4 典型应用场景

- **设置控件的背景色和前景色**：可以为窗口、按钮、文本输入框等控件设置背景色和文本颜色。
- **自定义控件的选中颜色**：可以自定义文本选中时的高亮颜色。
- **支持不同的控件状态**：可以根据控件的状态（如正常、禁用、选中等）设置不同的颜色。

2.5 QBrush

在 PyQt5 中，`QBrush` 是一个用于定义填充图案的类。它属于 `PyQt5.QtGui` 模块，主要用于设置控件、图形或其他绘图元素的填充样式。通过 `QBrush`，你可以为图形、文本、背景等定义颜色、渐变、纹理等填充效果。

`QBrush` 的主要作用是定义填充样式，通常用于以下场景：

- **设置图形的填充颜色**：如矩形、椭圆等图形的填充颜色。
- **定义渐变填充**：可以使用线性渐变、径向渐变等复杂的填充效果。
- **使用纹理填充**：可以通过图片或图案来填充图形。
- **设置透明度**：可以为填充样式设置透明度。

2.5.1 `QBrush` 的导入方式

```
from PyQt5.QtGui import QBrush
```

2.5.2 常用方法

2.5.2.1 创建 `QBrush` 对象

- `QBrush(QColor color)`：通过颜色创建一个 `QBrush` 对象。

```
from PyQt5.QtGui import QBrush, QColor

brush = QBrush(QColor(255, 0, 0))  # 创建一个红色的画刷
```

- `QBrush(Qt.BrushStyle style)`：通过填充样式创建一个 `QBrush` 对象。
  - 常用的填充样式有：
    - `Qt.SolidPattern`：纯色填充（默认）。
    - `Qt.Dense1Pattern` 到 `Qt.Dense7Pattern`：不同的密度点阵填充。
    - `Qt.HorPattern`：水平线条填充。
    - `Qt.VerPattern`：垂直线条填充。
    - `Qt.CrossPattern`：十字交叉线条填充。

华清远见开源实验中心 yyzlab.com.cn

- `Qt.BDiagPattern`：反斜线填充。
- `Qt.FDiagPattern`：正斜线填充。
- `Qt.DiagCrossPattern`：交叉斜线填充。
- `Qt.NoBrush`：无填充。

```
from PyQt5.QtCore import Qt

brush = QBrush(Qt.Dense4Pattern)  # 创建一个密集点阵填充的画刷
```

2.5.2.2 设置渐变填充

- `QBrush(QGradient gradient)`：通过渐变对象（如 `QLinearGradient`、`QRadialGradient` 等）创建一个 `QBrush` 对象。

```
from PyQt5.QtGui import QBrush, QLinearGradient

gradient = QLinearGradient(0, 0, 100, 100)
gradient.setColorAt(0, QColor(255, 0, 0))  # 起始颜色为红色
gradient.setColorAt(1, QColor(0, 0, 255))  # 结束颜色为蓝色

brush = QBrush(gradient)  # 使用渐变作为填充
```

2.5.2.3 设置纹理填充

- `QBrush(QPixmap pixmap)`：通过图片创建一个 `QBrush` 对象，用于纹理填充。

```
from PyQt5.QtGui import QBrush, QPixmap

brush = QBrush(QPixmap("texture.png"))  # 使用图片作为填充纹理
```

2.5.2.4 设置透明度

- `setColor(QColor color)`：设置画刷的颜色，并可以通过 `QColor` 设置透明度。

```
from PyQt5.QtGui import QBrush, QColor

brush = QBrush()
brush.setColor(QColor(255, 0, 0, 128))  # 设置半透明的红色
```

2.5.2.5 获取画刷的属性

- `color()`：获取画刷的颜色。

```
color = brush.color()
print(f"画刷颜色: {color.name()}")
```

- `style()`：获取画刷的填充样式。

```
style = brush.style()
print(f"画刷样式: {style}")
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.5.2.6 将 `QBrush` 应用于绘图

- `QPainter.setBrush(QBrush)`：在绘图时使用 `QBrush` 来设置填充样式。

```
from PyQt5.QtWidgets import QWidget
from PyQt5.QtGui import QPainter, QBrush, QColor

class BrushExample(QWidget):
    def paintEvent(self, event):
        painter = QPainter(self)
        brush = QBrush(QColor(255, 0, 0))
        painter.setBrush(brush)
        painter.drawRect(50, 50, 100, 100)  # 绘制一个红色填充的矩形
```

2.5.3 典型应用场景

- **设置图形的填充颜色**：可以为矩形、椭圆等图形设置填充颜色。
- **定义渐变填充**：可以使用线性渐变、径向渐变等复杂的填充效果。
- **使用纹理填充**：可以通过图片或图案来填充图形。
- **设置透明度**：可以为填充样式设置透明度，增强视觉效果。

华清远见元宇宙实验中心
yyzlab.com.cn